FOLHA 1 — FRENTE Códigos de Sistemas Operacionais (P1 / AT01: Capítulos 01 a 09)

Syscalls C/ASM, TCB Linux, Processos, Pthreads, POSIX Queues

Cap 02 Invocação de uma syscall (em C) (Slide 18/22)

```c
#include <unistd.h>
int main (int argc, char *argv[])
{
    write (1, "Hello World!\n", 13) ;    /* write string to stdout */
    _exit (0) ;                          /* exit with no error */
}
```

Cap 02 Invocação de uma syscall (assembly) (Slide 19...

```assembly
section .data
    msg db 'Hello World!', 0xA       ; output string (0xA = "\n")
    len equ 13                       ; string size
section .text
global _start
_start:
    mov rax, 1                       ; syscall opcode (1: write)
    mov rdi, 1                       ; file descriptor (1: stdout)
    mov rsi, msg                     ; data to write
    mov rdx, len                     ; number of bytes to write
    syscall                          ; make syscall
    mov rax, 60                      ; syscall opcode (60: _exit)
    mov rdi, 0                       ; exit status (0: no error)
    syscall                          ; make syscall
```

Cap 05 TCB no Linux 1.0 (include/linux/sched.h) (Slide 6/29)

```c
struct task_struct {
    volatile long state;                /* -1 unrunnable, 0 runnable, >0
    long counter;
    long priority;
    unsigned long signal;
    unsigned long blocked;              /* bitmap of masked signals */
    unsigned long flags;                /* per process flags, defined be
    int errno;
    int debugreg[8];                    /* Hardware debugging registers
    ...
    struct task_struct *next_task, *prev_task;
    struct sigaction sigaction[32];
    int exit_code, exit_signal;
    ...
    unsigned long start_code, end_code, end_data, start_brk, brk,
    start_stack, start_mmap;
    unsigned long arg_start, arg_end, env_start, env_end;
    ...
    struct wait_queue *wait_chldexit;   /* for wait4() */
    unsigned short uid, euid, suid;
    unsigned short gid, egid, sgid;
    ...
    int tty;                            /* -1 if no tty, so it must be s
    unsigned short umask;
    struct inode * pwd;
    ...
    struct vm_area_struct *stk_vma;
};
```

Cap 05 Criação de processos (Slide 12/29)

```c
Processos são criados através de chamadas de sistema.
Em Windows:
CreateProcess: cria um novo processo, informando
o arquivo contendo o código a executar.
Em UNIX:
fork(): duplica o processo atual.
execve(file): substitui o código executável do processo
por outro código.
```

Cap 05 Criação de processos em UNIX (Slide 14/29)

```c
int main (int argc, char *argv[], char *envp[])
{
    int ret ;
    ret = fork () ;                     // replicação do processo
    if ( ret < 0 ) {
          // fork não funcionou
        perror ("Erro: ") ;
        exit (-1) ;
    }
else
    if ( ret > 0 )                      // sou o processo pai
    wait (0) ;                          // esperar filho concluir
else {
          // sou o processo filho
        execve ("/bin/date", argv, envp) ;   // carrega binário
        perror ("Erro: ") ;                  // execve não funcionou
    }
    printf ("Tchau !\n") ;
    exit(0) ;
}
```

Cap 05 Programando com threads em C (Slide 26/29)

```c
#include <pthread.h>
...
#define NUM_THREADS 5
void *print_hello (void *threadid)   // cada thread executa esta função
{
    printf ("%ld: Hello World!\n", (long) threadid);
    sleep (5) ;
    printf ("%ld: Bye bye World!\n", (long) threadid);
    pthread_exit (NULL);                // encerra esta thread
}
int main (int argc, char *argv[])    // thread "main"
{
    pthread_t thread[NUM_THREADS];
    long status, i;
    for(i = 0; i < NUM_THREADS; i++)     // cria as demais threads
    {
        printf ("Creating thread %ld\n", i);
        status = pthread_create (&thread[i], NULL, print_hello, (void *) i
        if (status) {
              // ocorreu um erro
            perror ("pthread_create");
            exit (-1);
        }
    }
    pthread_exit (NULL);                // encerra a thread "main"
}
```

Cap 05 Programando com threads em Java (Slide 27/29)

```java
public class MyThread extends Thread {
    int threadID;
    private static final int NUM_THREADS = 5 ;
    MyThread (int ID) {
        threadID = ID;
    }
    public void run () {
          // corpo de cada thread
        System.out.println (threadID + ": Hello World!") ;
        try {
            Thread.sleep(5000);
        }
    catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println (threadID + ": Bye bye World!") ;
    }
    public static void main (String args[]) {
        MyThread[] t = new MyThread[NUM_THREADS] ;
        for (int i = 0; i < NUM_THREADS; i++) {
              // cria as threads
            t[i] = new MyThread (i);
        }
        for (int i = 0; i < NUM_THREADS; i++) {
              // inicia a execução das threads
            t[i].start () ;
        }
    }
}
```

Cap 09 Filas POSIX - receptor (Slide 7/12)

```c
#define QUEUE "/my_queue"
mqd_t queue ;                           // descritor da fila de mensagens
struct mq_attr attr ;                   // atributos da fila de mensagens
int
msg ;                                   // as mensagens são números inteiros
  // abre ou cria a fila com permissoes 0666
if ((queue = mq_open (QUEUE, O_RDWR|O_CREAT, 0666, &attr)) < 0)
{
    perror ("mq_open") ;
    exit (1) ;
}                                       // recebe cada mensagem e imprime seu
for (;;)
{
    if ((mq_receive (queue, (void*) &msg, sizeof(msg), 0)) < 0)
    perror ("mq_receive:") ;
else
    printf ("Received msg value %d\n", msg) ;
}
```

Cap 09 Filas POSIX - emissor (Slide 8/12)

```c
#define QUEUE "/my_queue"           // abre a fila de mensagens, se exist
if ((queue = mq_open (QUEUE, O_RDWR)) < 0)
{
    perror ("mq_open") ;
    exit (1) ;
}
for (;;) {
    msg = random() % 100 ;              // valor entre 0 e 99
      // envia a mensagem
    if (mq_send (queue, (void*) &msg, sizeof(msg), 0) < 0)
    {
        perror ("mq_send") ;
        exit (1) ;
    }
    printf ("Sent message with value %d\n", msg) ;
    sleep (1) ;
}
```

FOLHA 1 — VERSO Teoria do Escalonador, Fórmulas Algébricas e Algoritmos Cooperativos

Como Desenhar sem Errar • FCFS e SJF Passo a Passo

O Modelo Mental do Maziero e as Fórmulas Invariantes

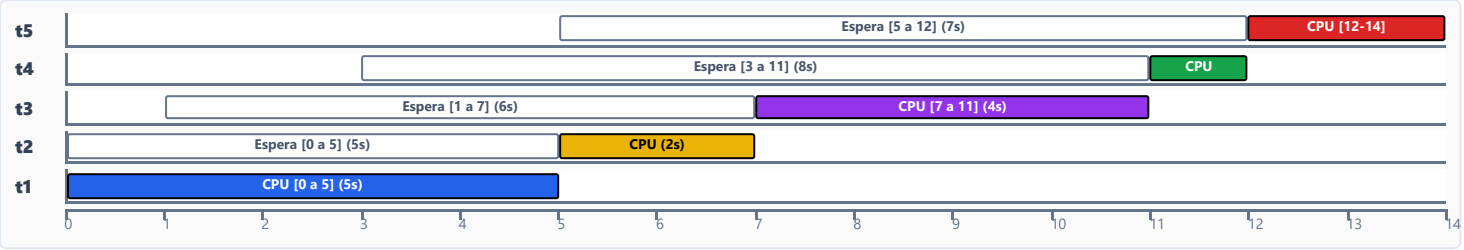
Não conte quadradinhos vazios para achar espera!

Fórmulas Matemáticas:
- **Tempo de Retorno / Vida (Turnaround):** $t_t(i) = f_i - c_i$ (fim menos chegada).
- **Tempo de Espera (Waiting):** $t_w(i) = t\ t(i) - d\ i = f_i - c_i - d_i$ (fim - chegada - duração).
- **Tempo de Resposta (Response):** $t_r(i) = p_i - c_i$ (primeira execução menos chegada).
- **Médias Gerais:** T_t = ∑ t_t / N | T_w = ∑ t_w / N.

O Exemplo Canônico do Capítulo 06:
- **t1:** Ingresso 0 | Dur 5 | Prio 2 (Azul)
- **t2:** Ingresso 0 | Dur 2 | Prio 3 (Amarelo)
- **t3:** Ingresso 1 | Dur 4 | Prio 1 (Roxo)
- **t4:** Ingresso 3 | Dur 1 | Prio 4 (Verde)
- **t5:** Ingresso 5 | Dur 2 | Prio 5 (Vermelho)

1. Escalonamento FCFS (First Come, First Served)

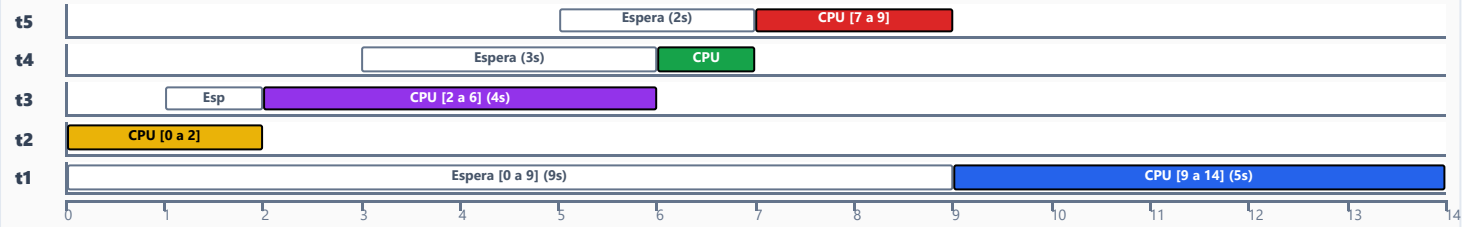
Tt = 8,0 s | Tw = 5,2 s | 4 Trocas



Passo a Passo de Desenho e Contas (FCFS):
1. Ordene estritamente por chegada: t1(0) → t2(0) → t3(1) → t4(3) → t5(5).
2. t1 roda [0 a 5] ⇒ f1 = 5, tt = 5-0 = 5s, tw = 5-5 = 0s.
3. t2 espera de 0 a 5 e roda [5 a 7] ⇒ f2 = 7, tt = 7-0 = 7s, tw = 7-2 = 5s.
4. t3 espera de 1 a 7 e roda [7 a 11] ⇒ f3 = 11, tt = 11-1 = 10s, tw = 10-4 = 6s.
5. t4 espera de 3 a 11 e roda [11 a 12] ⇒ f4 = 12, tt = 9s, tw = 8s. | t5 espera [5 a 12] e roda [12 a 14] ⇒ f5 = 14, tt = 9s, tw = 7s.

2. Escalonamento SJF (Shortest Job First - Cooperativo)

Tt = 5,8 s | Tw = 3,0 s | 4 Trocas



Passo a Passo de Desenho e Decisões em Cada Término (SJF):
- **t = 0:** Prontas {t1(d=5), t2(d=2)}. Menor d é t2 ⇒ pinte t2 [0 a 2].
- **t = 2:** t2 acabou. Prontas {t1(5), t3(4)}. Menor d é t3 ⇒ pinte t3 [2 a 6].
- **t = 6:** t3 acabou. Prontas {t1(5), t4(1), t5(2)}. Menor d é t4 ⇒ pinte t4 [6 a 7].
- **t = 7:** t4 acabou. Prontas {t1(5), t5(2)}. Menor d é t5 ⇒ pinte t5 [7 a 9].
- **t = 9:** Só resta t1 ⇒ pinte t1 [9 a 14]. Conclusões: f = [14, 2, 6, 7, 9] ⇒ Tw = [9, 0, 1, 3, 2] ⇒ Média Tw = 3,0 s.

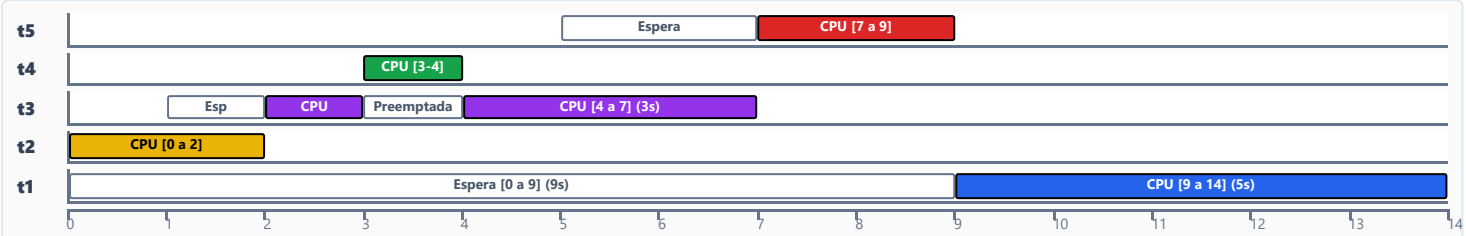
FOLHA 2 — FRENTE

Escalonadores Preemptivos: SRTF e Round-Robin (Capítulo 06)

Preempção por Menor Restante • O Diagrama da Fila do Slide 11 Passo a Passo

3. Escalonamento SRTF (Shortest Remaining Time First)

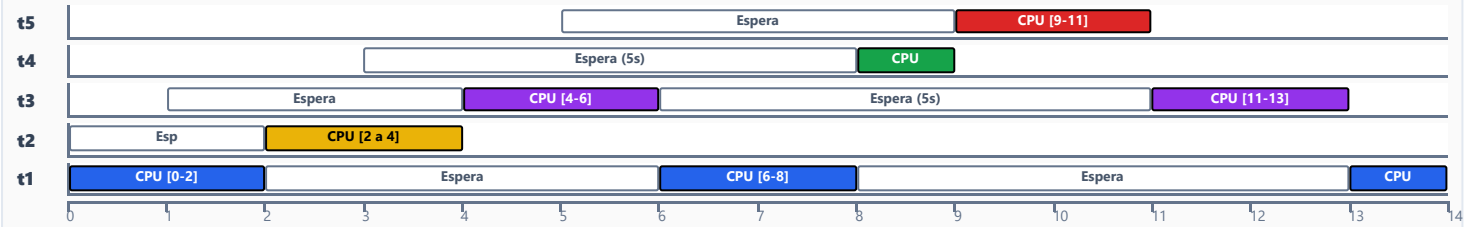
Tt = 5,4 s (Mínimo) | Tw = 2,6 s | 5 Trocas



O Ponto Crítico de Desenho (Preempção em t = 3):
• Em t = 2, t3 começa a rodar. Em t = 3, chega t4(d=1). Resta de t3 = 3 s. Como 1 < 3, **t4 preempta t3**!
• *No desenho:* **corte a barra colorida de t3 em 3**, pinte t4 em [3-4], deixe caixa branca de espera em t3 em [3-4], e retome t3 em 4!

4. Escalonamento Round-Robin (Revezamento com Quantum tq = 2) — O Slide 11 Passo a Passo

Tt = 8,4 s | Tw = 5,6 s | 7 Trocas (Máximo)

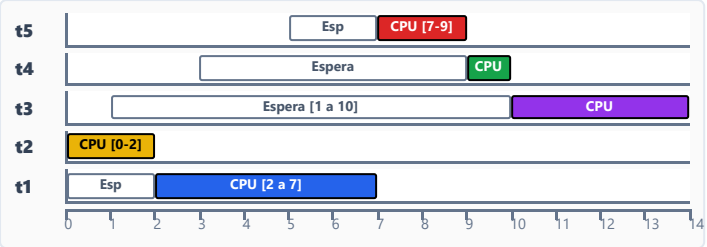


Tempo	Fila Prontos	CPU	Evento / Transição Didática	Tempo	Fila Prontos	CPU	Evento / Transição Didática
t = 0	[t2]	t1	t1 entra na CPU; t2 aguarda na fila	t = 8	[t1, t3, t5]	t4	t4 roda 1 s e conclui
t = 1	[t3, t2]	t1	t3 chega e entra no fim da fila	t = 9	[t1, t3]	t5	t4 conclui (FIM); t5 assume a CPU
t = 2	[t1, t3]	t2	Fim do quantum de t1; t2 assume a CPU	t = 10	[t1, t3]	t5	t5 completa seu 2º segundo de fatia
t = 3	[t4, t1, t3]	t2	t4 chega e entra no fim da fila	t = 11	[t1]	t3	t5 conclui (FIM); t3 assume a CPU
t = 4	[t4, t1]	t3	t2 conclui (FIM); t3 assume a CPU	t = 12	[t1]	t3	t3 completa seu 2º segundo de fatia
t = 5	[t5, t4, t1]	t3	t5 chega e entra no fim da fila	t = 13	[vazio]	t1	t3 conclui (FIM); t1 roda seu 1 s final
t = 6	[t3, t5, t4]	t1	Fim do quantum de t3; t1 assume a CPU	t = 14	[vazio]	—	t1 conclui (FIM). Todas finalizadas!
t = 7	[t3, t5, t4]	t1	t1 completa seu 2º segundo na CPU	**Regra: Novas chegadas entram ANTES de quem sofre preempção!**			

Prioridades, Aging, Quadro Comparativo & Inversão de Prioridade

5. PRIOc (Prioridade Cooperativa)

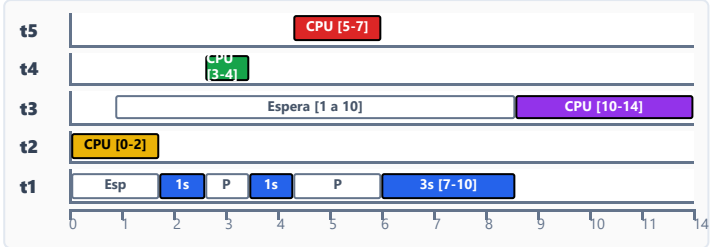
Tt=6,6s | Tw=3,8s



Ordem: t2(p=3) → t1(p=2) → t5(p=5) → t4(p=4) → t3(p=1). Como é cooperativo, ninguém interrompe t1 em 3 ou 5.

6. PRIOp (Prioridade Preemptiva)

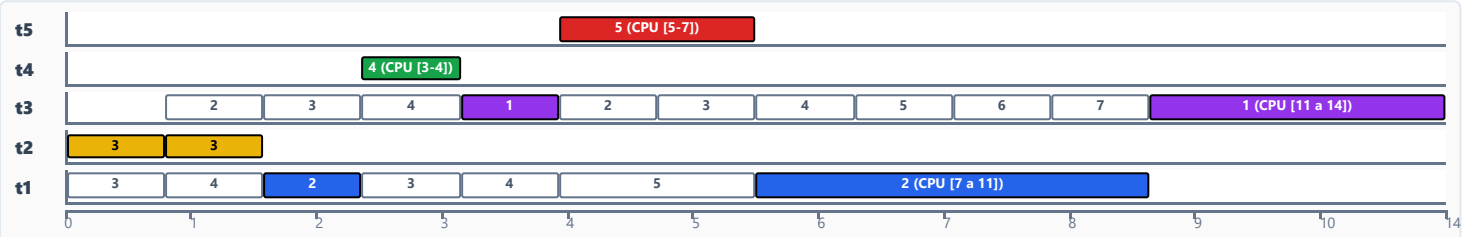
Tt=5,6s | Tw=2,8s



Duas preempções em t1: Cortada em t=3 por t4(p=4) e em t=5 por t5(p=5). Barra de t1 fatiada em: [2-3], [4-5] e [7-10].

7. Escalonamento PRIOd (Prioridade Dinâmica / Aging com α = 1) — Fiel ao Slide 23

Tt = 5,8 s | Tw = 3,0 s | 6 Trocas



O Segredo dos Números no Desenho (Slide 23): Anote a prioridade dinâmica (pd) em cada segundo!

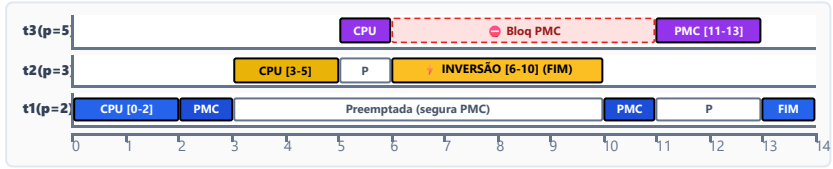
- Se espera na fila: **pd ← pd + 1**. Se executa na CPU: reseta para a prioridade estática original **pd ← pe**.
- Em t = 4, t3 esperou 3s e sua prioridade subiu para 4 (igual a t1); desempata e ganha a CPU, evitando inanição (starvation)!

Quadro Comparativo Oficial (Slide 26)

Algoritmo	Tt médio	Tw médio	Trocas
FCFS	8,0 s	5,2 s	4
RR (q=2)	8,4 s	5,6 s	**7 (Máx)**
SJF	5,8 s	3,0 s	4
SRTF	**5,4 s (Mín)**	**2,6 s (Mín)**	**5**
PRIOc	6,6 s	3,8 s	4
PRIOp	5,6 s	2,8 s	6
PRIOd	5,8 s	3,0 s	6

Questão de Prova: Inversão de Prioridade & PMC

25 Pontos Moodle



Campo da Questão na Prova	Instante	Motivo / Dedução Direta
t1 solicita acesso exclusivo a PMC:	**2**	Dado explícito no enunciado da questão.
t1 recebe acesso exclusivo a PMC:	**2**	PMC estava livre; ganha de imediato.
t1 libera a PMC:	**11**	Executa 1s em [2-3], é preemptada por t2, retoma em 10 e roda +1s em [10-11].
t3 solicita acesso exclusivo a PMC:	**6**	Inicia em t=5; solicita 1 unidade após o início (5 + 1 = 6).
t3 recebe acesso exclusivo a PMC:	**11**	Fica bloqueada de 6 a 11; acorda no momento que t1 libera em 11.
t3 libera a PMC:	**13**	Roda sua seção crítica de 2 unidades inteiramente em [11-13] (11 + 2 = 13).
t2 termina sua execução:	**10**	Roda 2s em [3-5], retoma em 6 (inversão enquanto t3 bloqueia) e conclui em 10.